

## Вопрос 1

Пока нет ответа

Балл: 1,00

Что такое placement new и где он используется?

Выберите один или несколько ответов:

- ☐ a. Выделяет новую память в куче
- ☐ b. Работает только с встроенными типами
- ☒ c. Используется в аллокаторах в методе `construct()`
- ☒ d. Используется в реализации `emplace_back` контейнеров
- ☒ e. Создание объекта в заранее выделенной памяти

## Вопрос 2

Пока нет ответа

Балл: 1,00

Что позволяют делать structured bindings?

Выберите один или несколько ответов:

- ☒ a. Работает с `std::pair`, `std::tuple`, агрегатными структурами
- ☒ b. Распаковка пар и структур в отдельные переменные
- ☐ c. Динамическое связывание переменных
- ☒ d. Упрощает обход `map`: `for (const auto& [key, value] : map)`
- ☐ e. Создание анонимных структур

### Вопрос 3

Пока нет ответа

Балл: 1,00

Какие утверждения о `decltype` верны?

Выберите один или несколько ответов:

- ☒ a. Используется с trailing return type (`-> decltype(...)`)
- ☒ b. Получение типа выражения без его вычисления (C++11)
- ☒ c. `decltype(auto)` для точного вывода типа выражения (C++14)
- ☐ d. `decltype` вычисляет выражение во время выполнения
- ☐ e. `decltype` заменяет `sizeof`

## Вопрос 4

Пока нет ответа

Балл: 1,00

Какие рекомендации дают эксперты по работе с контейнерами?

Выберите один или несколько ответов:

- ☐ a. Всегда используйте `list` для частых вставок
- ☒ b. Предпочитайте `vector` другим контейнерам
- ☒ c. Понимайте гарантии сложности операций
- ☒ d. Знайте, когда итераторы инвалидируются
- ☐ e. Никогда не используйте `reserve()`
- ☒ f. Используйте `policy-based design` для гибких контейнеров

## Вопрос 5

Пока нет ответа

Балл: 1,00

Какие утверждения о ключевом слове `auto` верны?

Выберите один или несколько ответов:

- ☐ a. `auto` означает динамическую типизацию
- ☐ b. `auto` работает только с встроенными типами
- ☒ c. Автоматический вывод типа переменной во время компиляции
- ☒ d. Упрощает код с итераторами (`auto it = vec.begin()`)
- ☒ e. Обязательная инициализация при объявлении

## Вопрос 6

Пока нет ответа

Балл: 1,00

Какие преимущества дает `std::string_view` (C++17)?

Выберите один или несколько ответов:

- ☒ a. Может принимать `const char*`, `std::string`, `std::string_view`
- ☒ b. Работа со строками без копирования данных
- ☐ c. Динамически выделяет память
- ☐ d. Владеет копией строковых данных

## Вопрос 7

Пока нет ответа

Балл: 1,00

Какие возможности предоставляет библиотека ranges в C++20?

Выберите один или несколько ответов:

- ☒ a. Выразительные цепочки операций с использованием оператора `|`
- ☒ b. Ленивые вычисления (элементы вычисляются при итерации)
- ☐ c. Замена всех контейнеров STL
- ☐ d. Только для работы с `vector`
- ☒ e. `views::filter`, `views::transform` и другие адаптеры

## Вопрос 8

Частично правильный

Баллов: 0,67 из 1,00

Какие компоненты STL используются в реализации LRU Cache?

Выберите один или несколько ответов:

☒ a. `std::unordered_map` для быстрого поиска итераторов в списке ☒

☐ b. `std::map` для хранения пар ключ-значение

☐ c. `std::vector` для хранения данных

☐ d. Perfect forwarding в методе `insert` (универсальные ссылки)

☒ e. `std::list` для хранения пар ключ-значение (поддержка `splice`) ☒



Ответ частично правильный.

Правильные ответы: `std::list` для хранения пар ключ-значение (поддержка `splice`), `std::unordered_map` для быстрого поиска итераторов в списке, Perfect forwarding в методе `insert` (универсальные ссылки)



## Вопрос 9

Пока нет ответа

Балл: 1,00

Какие утверждения о `std::optional` и `std::variant` верны?

Выберите один или несколько ответов:

- ☐ a. `std::optional` требует динамического выделения памяти
- ☒ b. `std::variant` может хранить значение одного из перечисленных типов
- ☐ c. `std::variant` — то же самое, что `union` в C
- ☒ d. `std::variant` — типобезопасное объединение (`union`)
- ☒ e. `std::optional` — значение, которое может присутствовать или отсутствовать

## Вопрос 10

Пока нет ответа

Балл: 1,00

Что верно об аллокаторах в C++?

Выберите один или несколько ответов:

- ☒ a. Позволяют использовать пулы памяти, сборщики мусора
- ☒ b. Стратегия выделения памяти для контейнеров
- ☐ c. Все контейнеры используют один стандартный аллокатор
- ☒ d. Методы: `allocate()`, `deallocate()`, `construct()`, `destroy()`
- ☒ e. `std::pmr::polymorphic_allocator` (C++17) для полиморфного управления памятью
- ☐ f. Аллокаторы отвечают только за выделение памяти через `new`